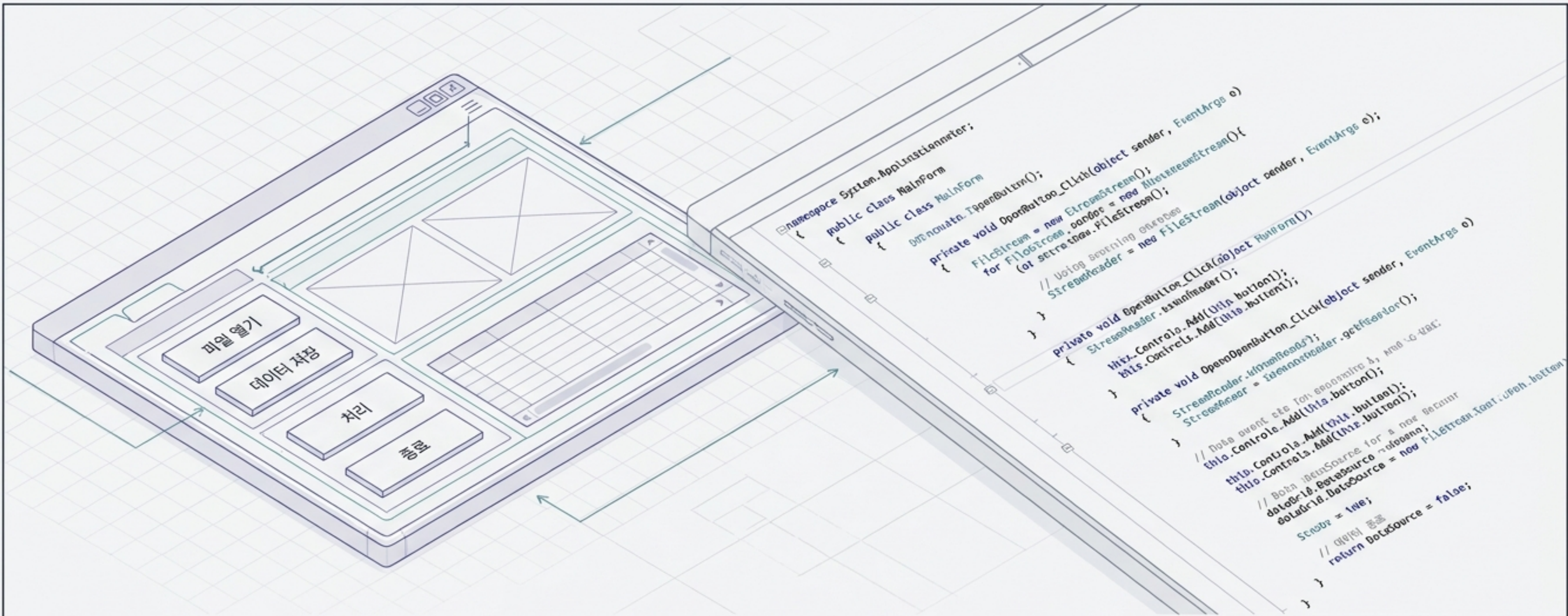




C# 개발자의 블루프린트: 데이터 흐름과 UI 설계

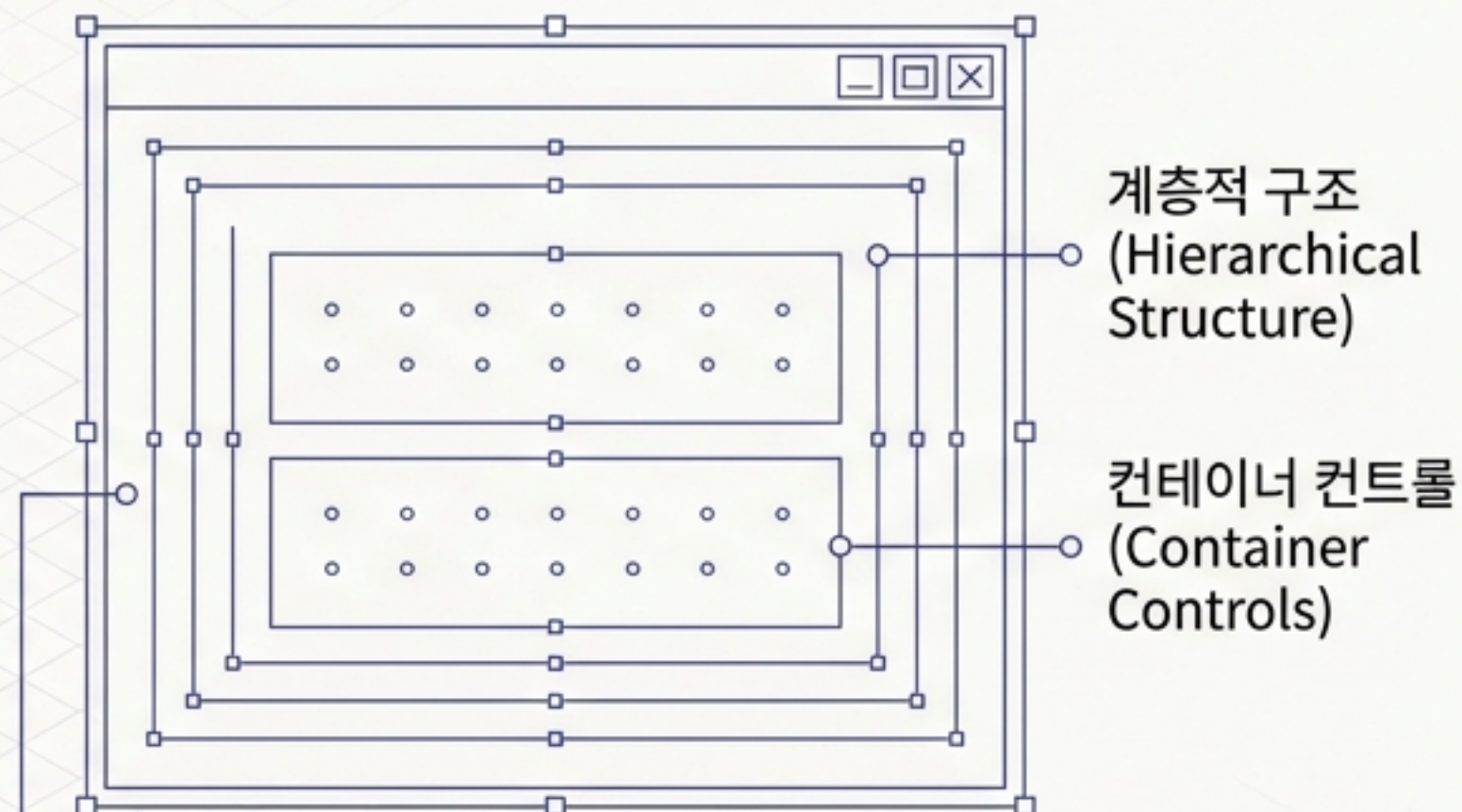
파일 스트림(I/O)부터 윈도우 폼(WinForms) 컨트롤 계층 구조까지, 애플리케이션 구축의 핵심 패턴 이해하기

1부. 데이터 관리 (로직)



텍스트 파일과 스트림(Stream)을 활용한
안전하고 효율적인 파일 입출력 제어.

2부. 인터페이스 관리 (프레젠테이션)



윈도우 폼 컨트롤의 공간적 격리와
이벤트 처리를 통한 사용자 상호작용 설계.

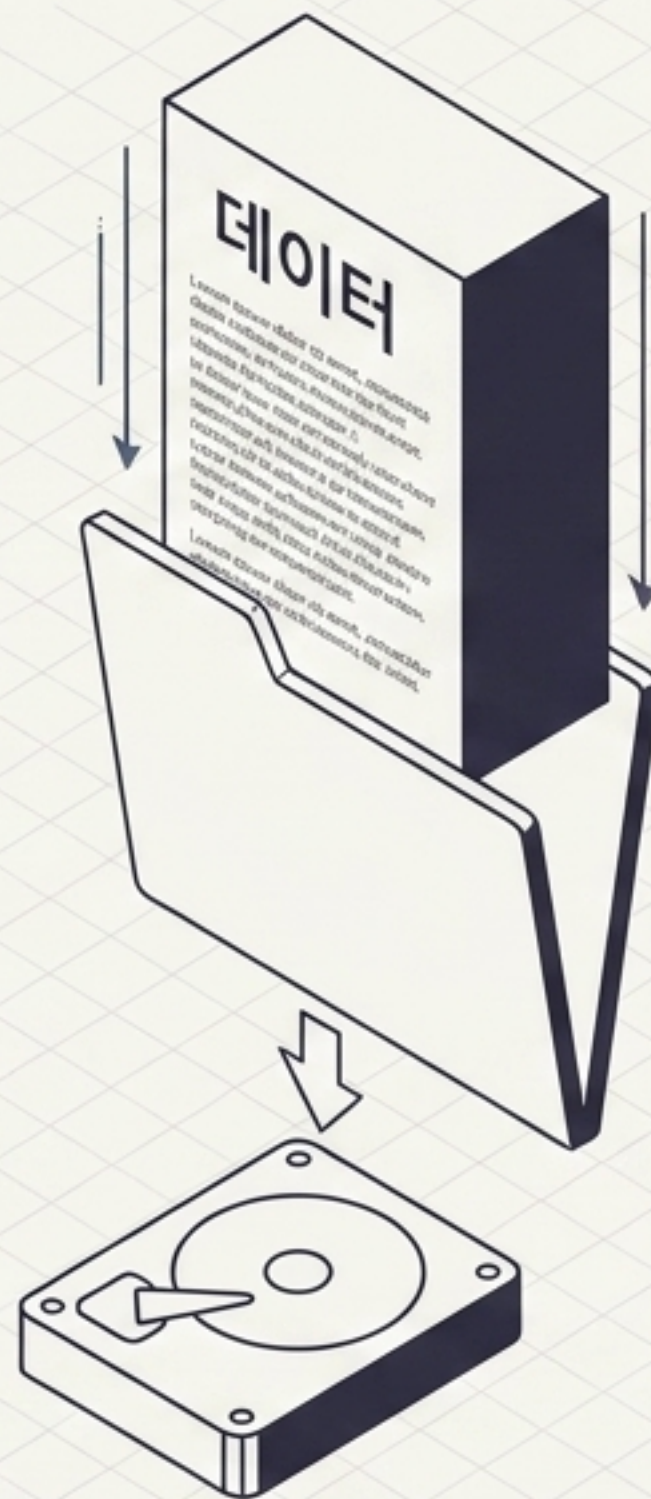
핵심 인사이트: C#의 모든 것은 '**객체(Object)**'입니다. 파일 스트림을 해제하거나 폼의 UI 컨트롤을 순회하는 것 모두 동일한 **논리적 패턴(반복문, 인스턴스 관리)**을 따릅니다.

가장 단순한 I/O: 한 번에 쓰고 읽기

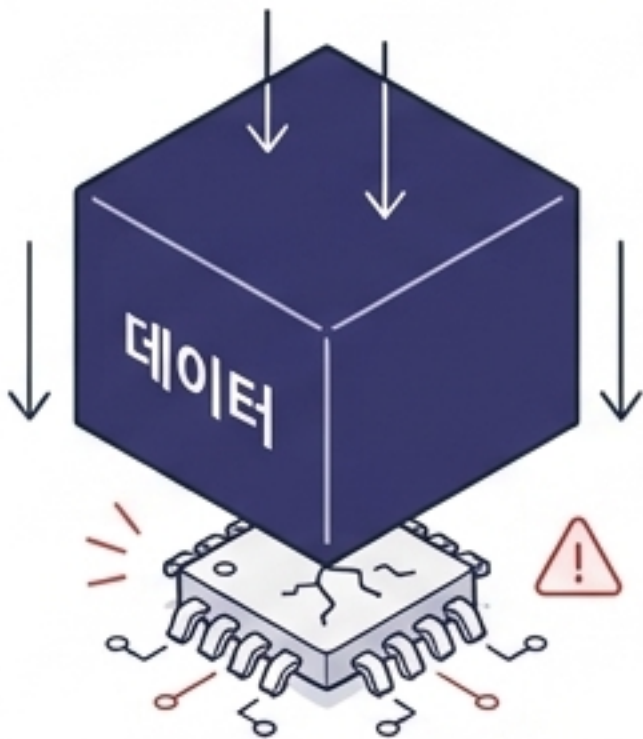
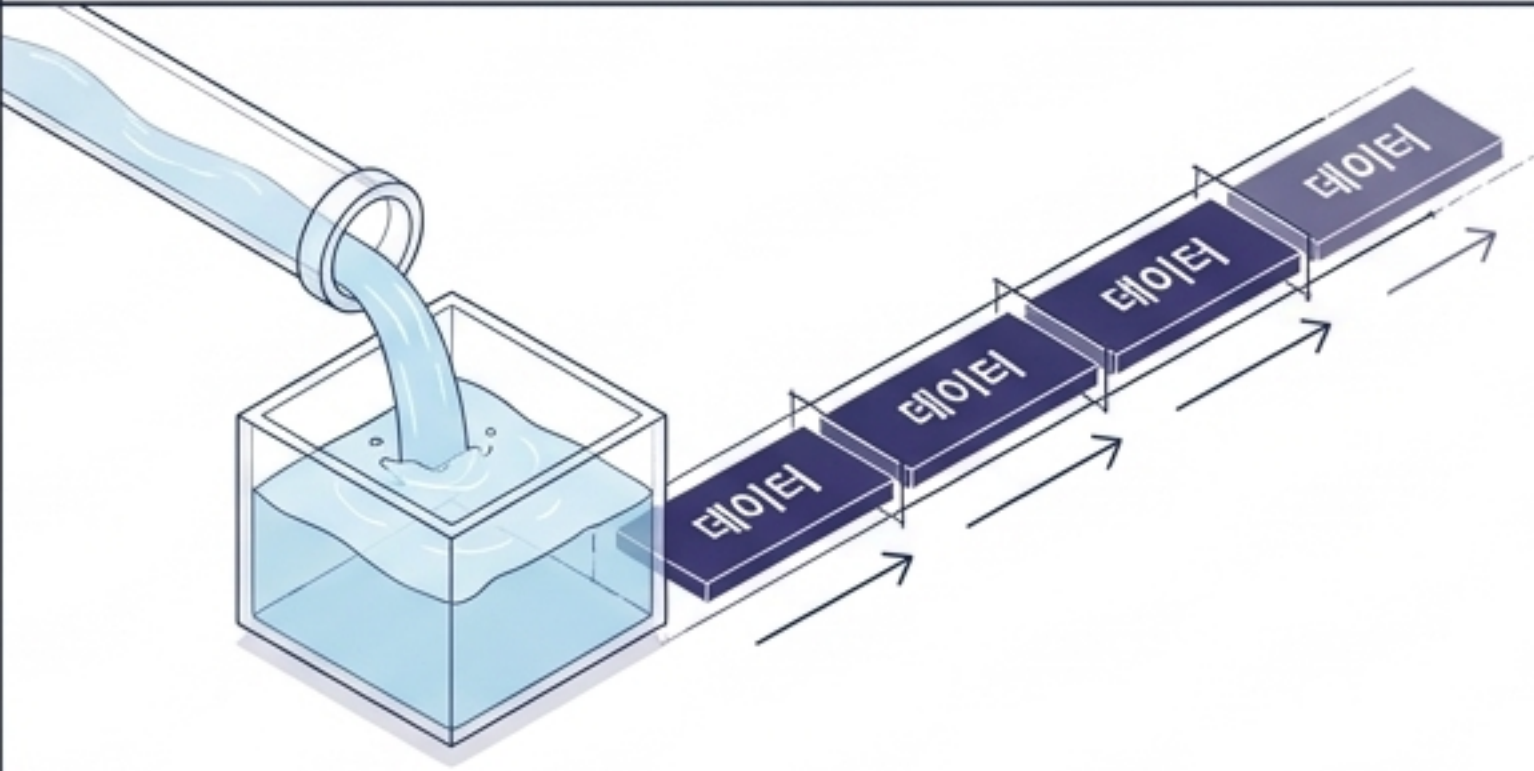
단일 메서드 호출로 파일을 열고,
쓰고, 닫습니다.
소용량 파일 처리에 적합합니다.

```
1 File.WriteAllText(@"c:\test\test.txt", "문자열을 파일에 씁니다.");  
2 Console.WriteLine(File.ReadAllText(@"c:\test\test.txt"));
```

축자 문자열(Verbatim String).
'@' 기호를 사용하면 역슬래시('\')를 두
번('\\\\') 쓸 필요 없이 파일 경로를
직관적으로 작성할 수 있습니다.



한계 극복: 왜 스트림(Stream)이 필요한가?

File.ReadAllText (정적 메서드)	StreamWriter / StreamReader
	
- 파일 전체를 한 번에 메모리에 적재 - 대용량 파일 처리 시 메모리 과부하 위험	- 데이터를 쪼개어 순차적으로 처리 (한 줄씩) - 대용량 파일도 안정적으로 처리 가능 - TextWriter 추상 클래스를 상속받음



Takeaway: 아주 큰 용량의 파일을 다룰 때는 파일이 시스템을 다운시키지 않도록 한 줄씩 처리하는 스트림이 필수적입니다.

스트림의 필수 원칙: 안전한 자원 해제 (using 블록)

```
1 using (StreamWriter writer = new StreamWriter(@"c:\test\test.txt"))
2 {
3     // 파일 쓰기 로직
4 } // <- 여기서 자동으로 Dispose() 호출
```

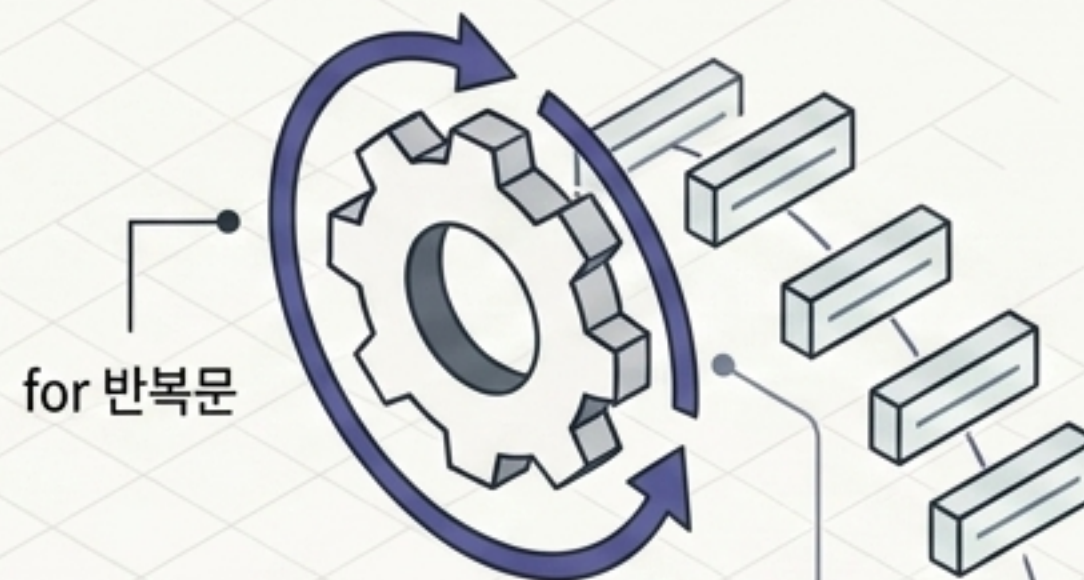
- 스트림 객체는 사용 후 반드시 메모리에서 해제해야 합니다.
- StreamWriter와 StreamReader는 IDisposable 인터페이스를 구현합니다.
- using 구문을 사용하면 중괄호 {} 블록을 탈출할 때 시스템이 자동으로 Dispose() 메서드를 호출하여 안전하게 파일 점유를 해제합니다.

Safe Containment



StreamWriter: 반복문을 통한 순차적 쓰기

```
01 writer.WriteLine("안녕하세요.");
02 for (int i = 0; i < 10; i++)
03 {
04     writer.WriteLine("반복문 - " + i);
05 }
```



```
안녕하세요.
StreamWriter 클래스를 사용해
글자를 여러 줄 입력해봅니다.
반복문 - 0
반복문 - 1
반복문 - 2
반복문 - 3
반복문 - 4
반복문 - 5
반복문 - 6
반복문 - 9
```

test.txt
(StreamWriter)

1

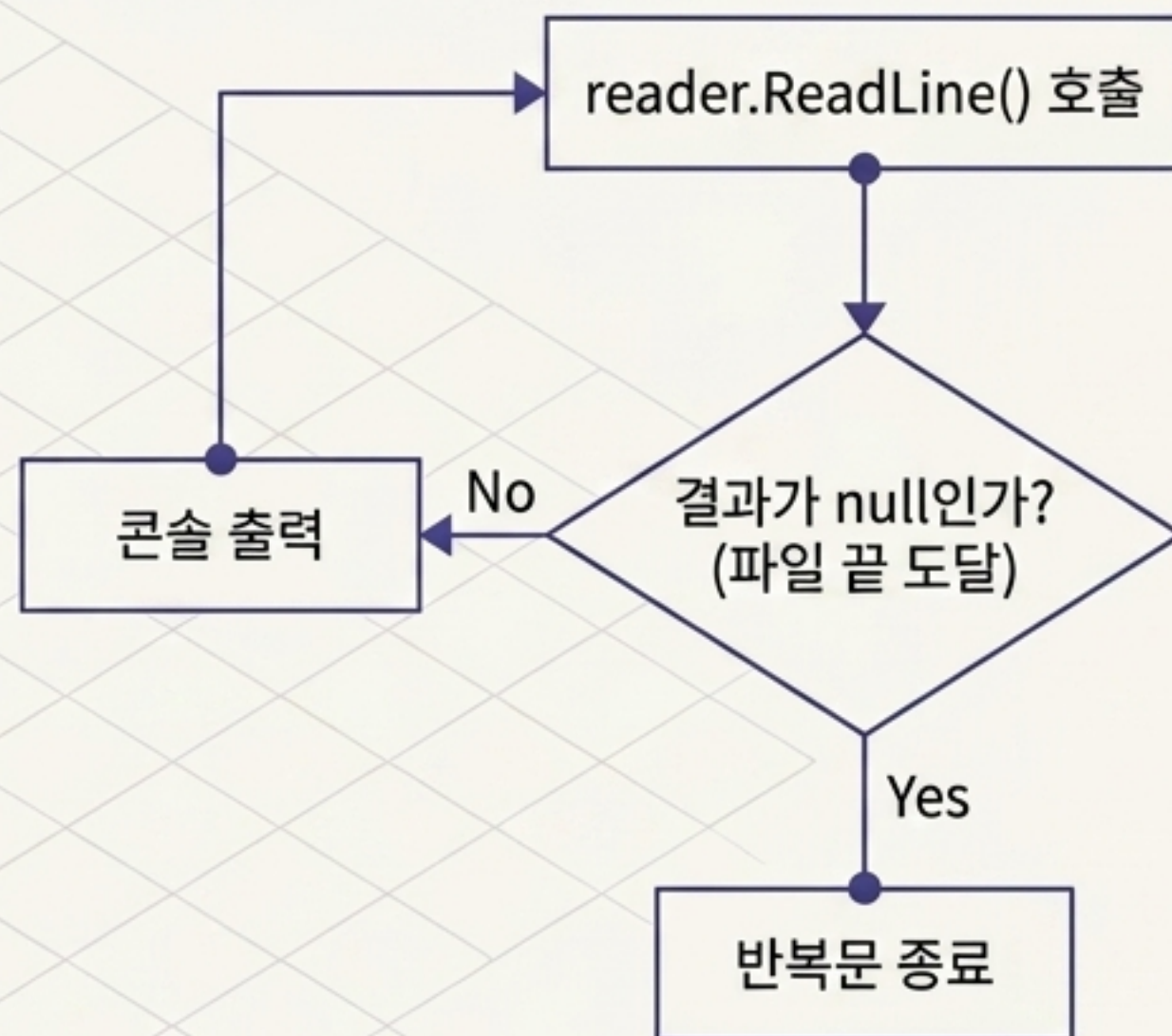
WriteLine(): 문자열을 입력하고 자동으로 줄바꿈(New Line)을 추가합니다.

2

for 반복문 결합: 메모리에 10줄을 한 번에 올리지 않고, I/O 버퍼를 통해 한 줄씩 디스크로 흘려보냅니다.

StreamReader: 끝을 만날 때까지 읽어오기

```
06 string line;  
07 while ((line = reader.ReadLine()) != null)  
08 {  
09     Console.WriteLine(line);  
10 }
```

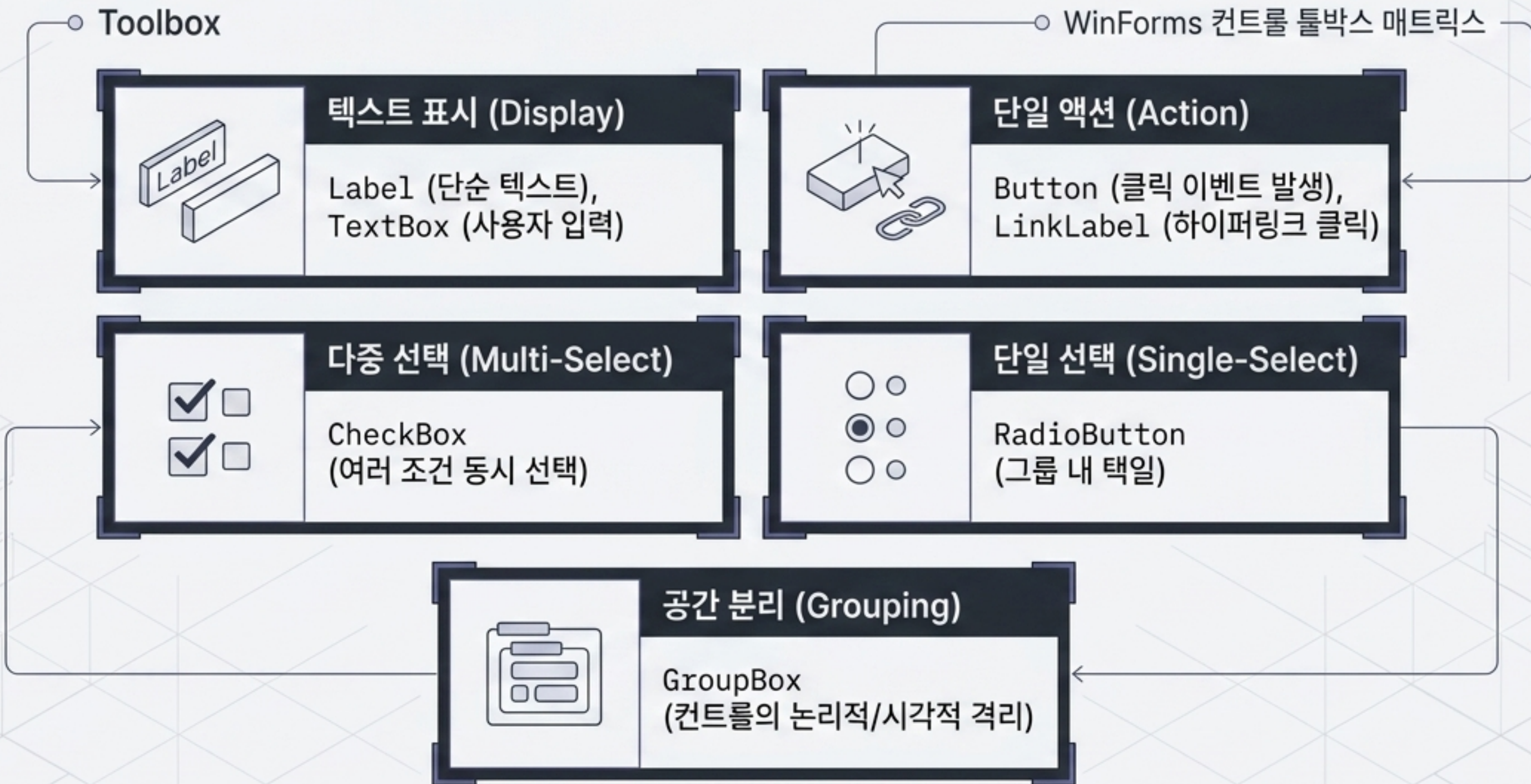


Takeaway: 파일의 끝(End of File)에 도달하면 `ReadLine()`은 `null`을 반환합니다. 이를 이용해 파일 용량에 관계없이 안전하게 전체 문서를 읽을 수 있습니다.



2부 시작: 윈도우 폼(WinForms) GUI 툴킷

UI 설계는 단순히 요소를 화면에 배치하는 것을 넘어, 사용자의 선택 방식을 제어하는 과정입니다.



정보 전달과 상호작용: Label vs LinkLabel

Label

글자 (Static Text)

Label: 단순 텍스트 출력용.

Text, Location, Height 속성으로 배치.

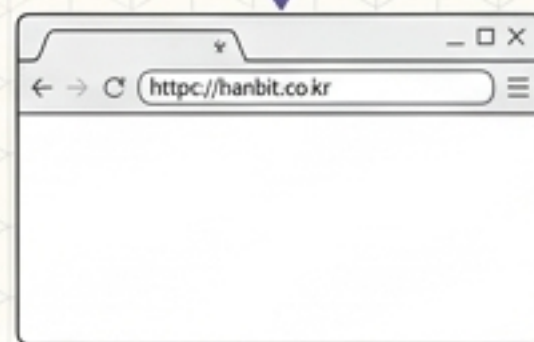
```
Label label = new Label() {  
    Text = "글자",  
    Location = new Point(10, 10),  
    Height = 23  
};
```

LinkLabel

글자 (Clickable Link)

LinkLabel: 클릭 이벤트를 지원하는 하이퍼링크.

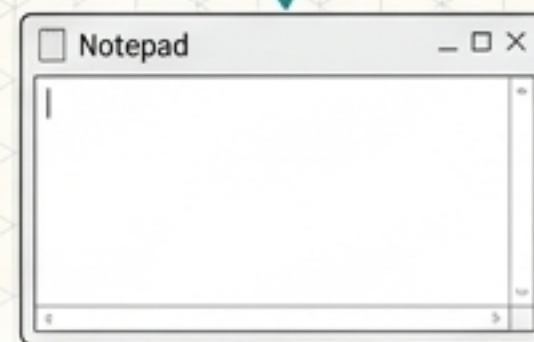
```
linkLabel.Click += LabelClick;
```



웹 브라우저 열기:

```
Process.Start("https://hanbit.co.kr");
```

```
System.Diagnostics.Process.Start("https://hanbit.co.kr");
```



외부 프로그램 실행:

```
Process.Start("notepad.exe");
```

```
System.Diagnostics.Process.Start("notepad.exe");
```

다중 선택 (CheckBox)과 컬렉션 순회 로직

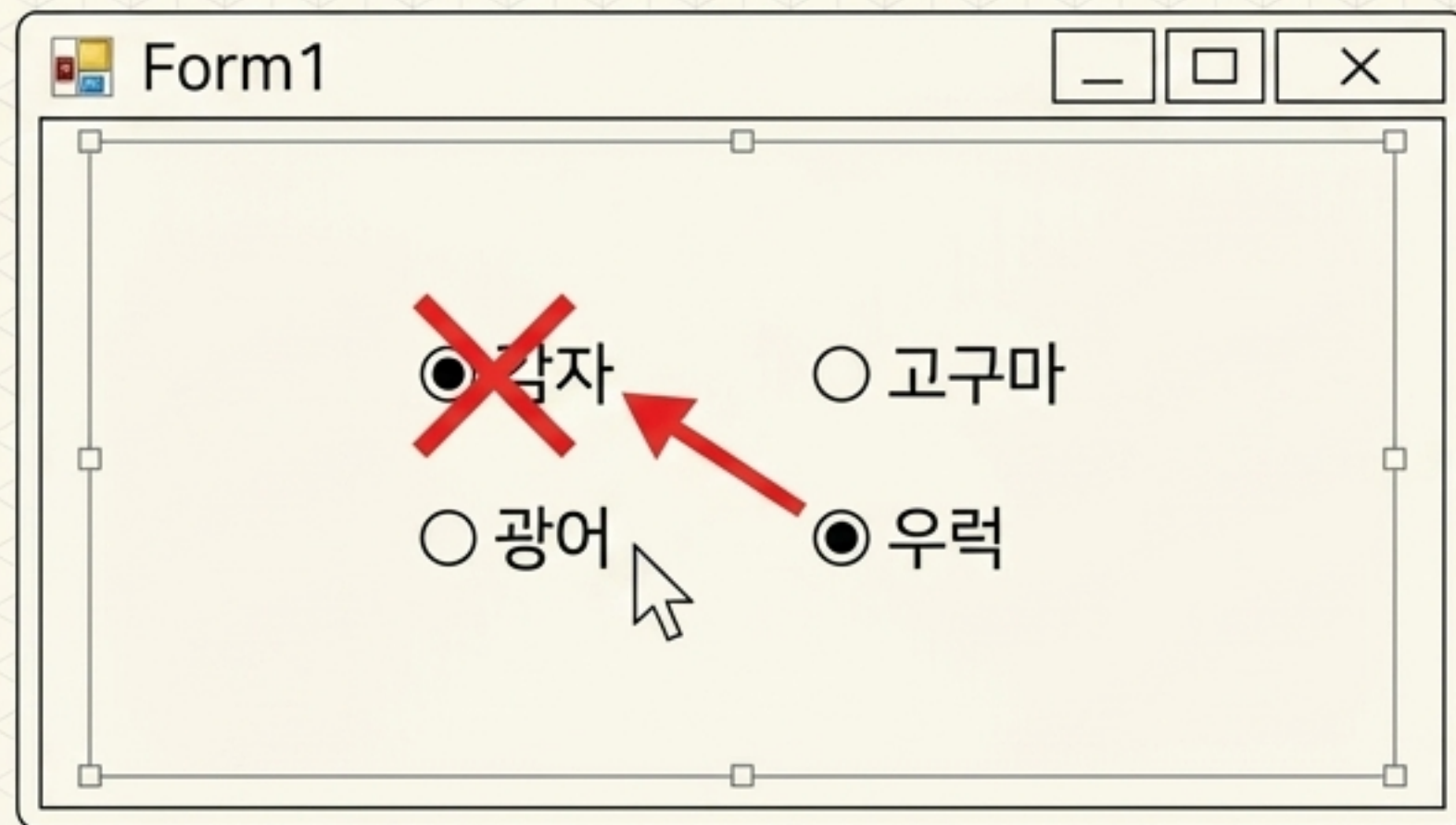


```
01 foreach (var item in Controls)
02 {
03     if (item is CheckBox) // 객체 타입 확인
04     {
05         CheckBox checkBox = (CheckBox)item; // 타입 캐스팅
05         if (checkBox.Checked) list.Add(checkBox.Text);
06     }
07 }
```

폼 위의 모든 요소는 Controls 리스트에 담깁니다.
반복문으로 요소를 순회하며, 해당 요소가 CheckBox 타입인지
확인(is)하고 변환(캐스팅)하여 선택 여부를 판별합니다.

라디오 버튼(RadioButton)의 딜레마

라디오 버튼은 한 번에 단 하나만 선택할 수 있는 상호배타적 컨트롤입니다.



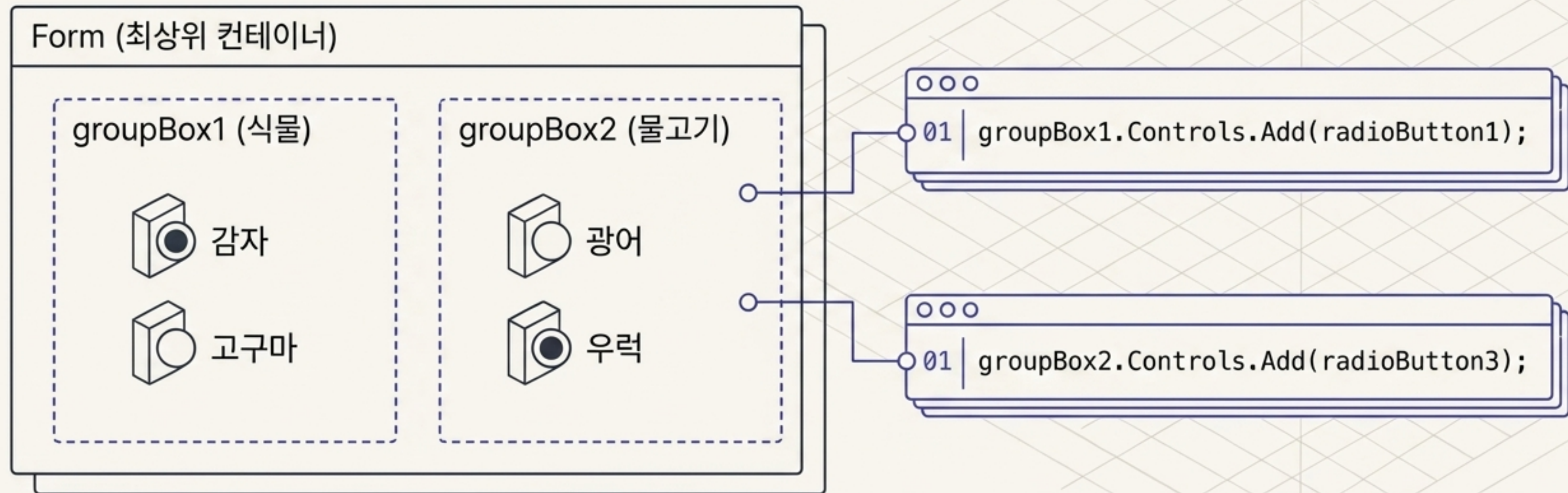
The Problem

폼(Form)이라는 단일 공간(Controls) 안에 라디오 버튼을 그냥 배치하면, 4개 중 단 1개만 선택할 수 있습니다.

그렇다면 여러 개의 독립적인 '단일 선택 그룹'을 만들려면 어떻게 해야 할까요?

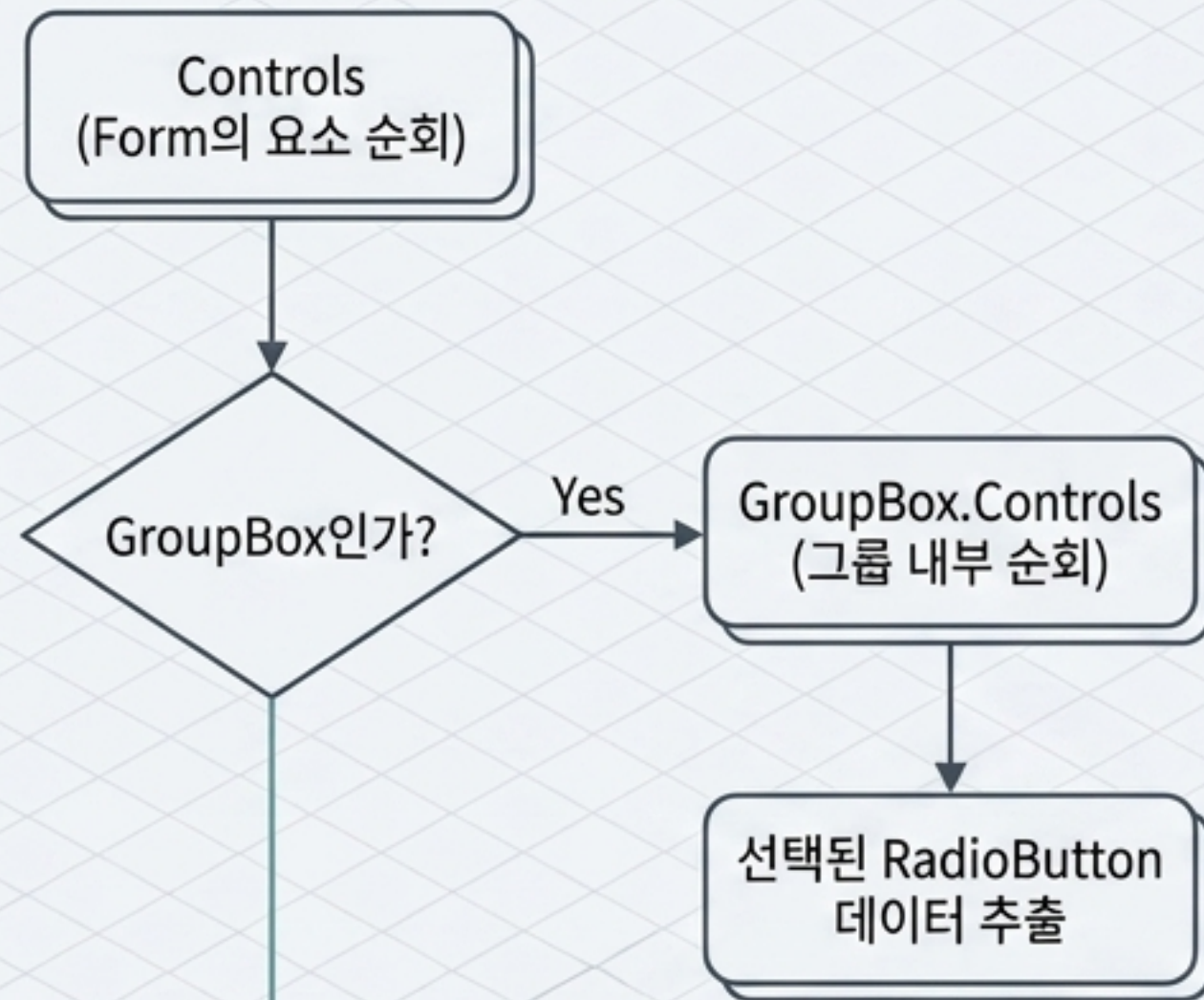
공간적 격리를 통한 논리적 그룹화: GroupBox

격벽 시스템: GroupBox는 컨트롤들을 논리적, 시각적으로 완전히 분리하는 컨테이너입니다.



Result: 폼 대신 그룹 박스의 Controls에 라디오 버튼을 추가하면, 각 그룹 박스가 독립적인 선택 영역으로 작동하여 각각 1개씩(총 2개) 선택이 가능해집니다.

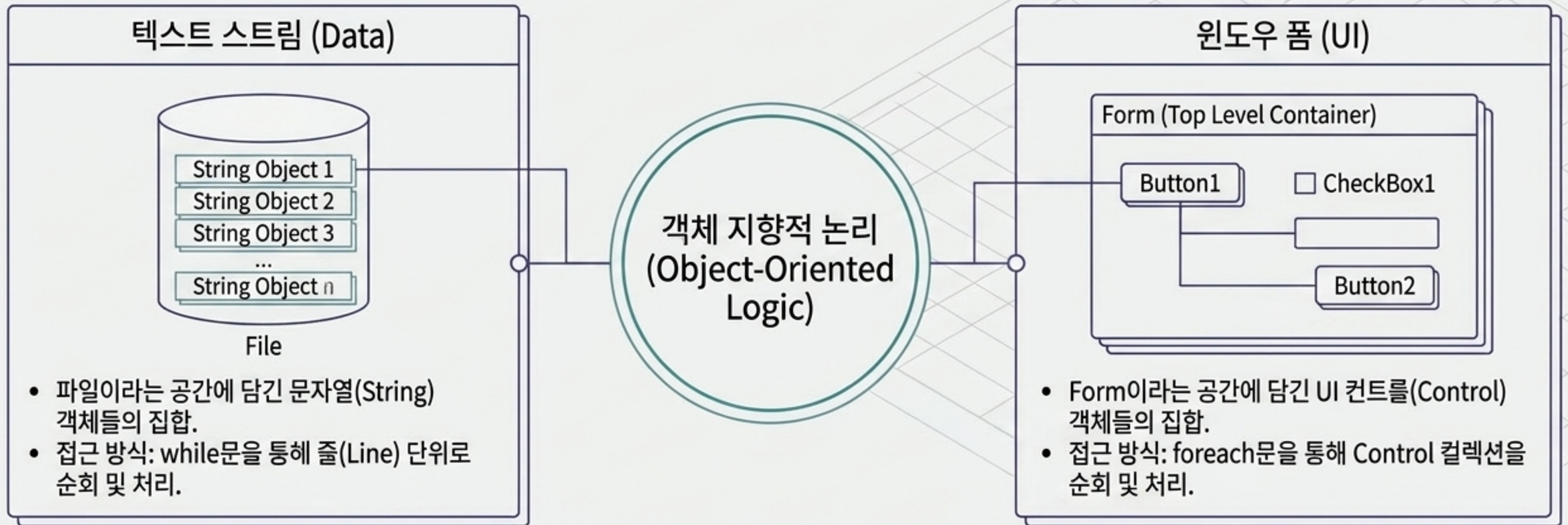
중첩 컬렉션(Nested Collections)의 데이터 추출



```
03 foreach (var outerItem in Controls)
04 {
05     if (outerItem is GroupBox)
06     {
07         // 그룹 박스 내부의 컨트롤을 다시 순회
08         foreach (var innerItem in ((GroupBox)outerItem).Controls)
09         {
10             RadioButton radioButton = innerItem as RadioButton;
11             if (radioButton != null && radioButton.Checked) { /* 처리 로직 */ }
12         }
13     }
14 }
15 }
```

UI가 계층 구조(Form -> GroupBox -> RadioButton)를 가지게 되었으므로, 데이터 탐색 로직 역시 외부반복문(Controls)과 내부 반복문(GroupBox.Controls)의 중첩 구조를 띠게 됩니다.

통합 인사이드: C# 프로그래밍의 일관성



결론: 화면에 보이는 버튼이든 보이지 않는 파일 데이터든, C#에서는 모두 동일한 객체 지향적 컬렉션 패턴으로 제어됩니다. 이 구조를 이해하면 어떤 기능이든 논리적으로 구축할 수 있습니다.